# Final Report: Number Guessing Game

## Project Description:

I have created a single player, terminal based number guessing game. The program prompts the player on a difficulty mode (determining the range of numbers), generates a random number, and provides feedback to the user for each attempt in the form of “Too high”, “Too low”, or a congratulatory message. A message appears after the game allowing the player to either continue a new round or end the game.

The program is meant to have simple and straightforward mechanics where the player is prompted to provide their input. The program also tracks the amount of attempts made by the player, with the game ending after a maximum of 10 attempts. Whether or not the player guesses the target number, alongside the message at the end of the round, a second message is displayed showing the guess history of that round.

## Approach and Logic/ Data Structures Used:

I mainly used while loops and conditional statements to develop the game. The main while loop is what keeps the game going. As long as the condition is met for the loop, the inside of the loop can keep going. Inside the loop is where the conditional if, elif and else statements are stored that help determine the difficulty of the game. The chosen difficulty determines whether or not the player has to guess numbers between 1 and 50, 1 and 100, or 1 and 200. Additionally, there are standalone statements that determine things like the target number (using random.randint(1, maximum value)), keep track of the guess history in a list, and store  a value for the amount of attempted guesses.

There is another while loop that processes the players guess. There are three outcomes from this:

1. The player guesses the target number correctly
2. The player does not guess the target number but has remaining attempts
3. The player runs out of attempted guesses (capped at 10 guesses in my game)

Every guess is processed using conditional statements. If the guess is greater than the target, the program prints “Too high”. If the guess is smaller than the target, the program prints “Too low”. Otherwise, the player guesses the correct value and a is presented with a congratulations! I avoided invalid inputs by using the method .isdigit(), to make sure negative numbers, letters, or other characters couldn't be entered.

The main data structure used in this program was a list, since I stored the guess history of that round into a list. I used the .append() method to add the guesses in the order that the player guesses them. This list is printed at the end of the round. As mentioned earlier, I used variables to track and store things like number of attempts, the target number, the players current guess, etc.

Challenges Faced:

The biggest challenge I faced was learning to use GitHub! I feel like it was a little difficult to navigate online, especially since different videos and different sources were providing me different answers. It ended up being pretty easy, and I found the GitHub UI to be pretty understandable. Another issue that I had, which is funnily visible in my GitHub Blame, is that I kept forgetting to add things into the program that were required. I kept finding myself having to go back to the Canvas description of my project to ensure that I was meeting all the requirements. For example, I didn't add in the attempt limit cap until way after. This wasn't an issue, and I didn't find myself having to revise the code heavily, but it was more of a concentration issue. I also had difficulties with time management, and I found myself doing the bulk of the project in the past few days.

There was a small logic  error that I was able to fix. In the difficulty selection I originally only printed an “Invalid Difficulty” for difficulties that aren’t easy/medium/hard. I added a continue statement so that the game does not crash from a user typing errors.

Possible Improvements:

This specific game was simple enough to where a lot of improvements can be made to make the game more interactive, more challenging, or more interesting. I think an easy addition to the game could be a scoring system or some kind of way to let players know how well they did. I think this scoring system can be complicated or simple. The simple version would be basing the scores on the amount of attempts out of ten that it took to guess. In other words, guessing in 1 attempt would result in a 100%, 2 guesses would result in a 90%, etc. A more complicated scoring system could develop a score based on how close each guess was to the target, alongside the attempted guesses. The base game did not require a scoring system, but I think it's a fun addition to the game.